Bölüm 4: Dosyalar ve İstisnalar

SE 422 · Introduction to Data Science · Konu özeti

Kaynak slayt destesi: week3_part3_Files_Exceptions
7 Kasım 2026

İçindekiler

Bölüm 4 · Dosyalar ve İstisnalar 3

Bölümün amacı 3

Konu anlatımı 3

Neden dosya? (Slayt 1–7) 3

Dosya nesnesi ve open (Slayt 8–11) 3

Yazmak ve okumak (Slayt 12–14) 4

Ekleme ve sayısal veri (Slayt 15–16) 4

Dosyayı döngüyle işlemek (Slayt 17–19) 5

Kayıtlar ve alanlar (Slayt 20–21) 5

İstisnalar ve traceback (Slayt 22–23) 6

try / except (Slayt 24–27) 6

else, finally ve işlenmeyen istisnalar (Slayt 28–31) 7

Sık yapılan hatalar 8

Kısa tekrar soruları 8

Cevaplar 8

Bölüm 4 · Dosyalar ve İstisnalar

Bölümün amacı

Program kapandığında değişkenlerdeki veri kaybolur. Bu bölüm veriyi **dosyada kalıcı olarak saklamayı** ve çalışma sırasında oluşan **hataları (istisnaları) zarifçe yönetmeyi** öğretir. Sonunda şunları yapabilmelisin:

- Metin dosyasını `open` ile açıp okumak, yazmak, sonuna eklemek ve kapatmak
- Sayıları dosyaya yazarken ve okurken dönüştürmek
- Dosyayı döngüyle satır satır işlemek; kayıt ve alan kavramlarını kullanmak
- `try / except / else / finally` ile istisnaları yakalamak

Konu anlatımı

Neden dosya? (Slayt 1–7)

Veriyi dosyaya kaydetmeye **yazmak**, dosyadan almaya **okumak** denir. Dosya kullanmanın üç adımı vardır: **aç → işle → kapat**.

- **Metin dosyası** veriyi metin olarak, **ikili (binary) dosya** ise metne çevrilmemiş biçimde tutar.
- **Sıralı erişimde** veri baştan sona okunur; **doğrudan erişimde** istenen konuma atlanabilir.

Dosya nesnesi ve open (Slayt 8–11)

`dosya = open(dosya_adi, mod)` dosyayı açar ve bir **dosya nesnesi** döndürür. Program dosyayla bu nesne üzerinden çalışır.

Mod	Anlamı
`'r'`	okuma; dosya yoksa hata
`'w'`	yazma; dosya varsa **içeriği silinir**, yoksa oluşturulur
`'a'`	ekleme; yazılanlar mevcut içeriğin **sonuna** gelir

Yol verilmezse dosya programla aynı klasörde aranır. Windows yolları için başa `r` konur: `open(r'C:\Users\data.txt')`; böylece ters eğik çizgiler kaçış karakteri sayılmaz.

Yazmak ve okumak (Slayt 12–14)

`write(string)` dosyaya yazar, `close()` kapatır. `read()` tüm içeriği tek string olarak, `readline()` bir satırı (`'\n'` dahil) okur. Dosya nesnesi bir **okuma konumu** tutar; her `readline` sıradaki satırı verir. Satır sonundaki `'\n'`, `rstrip('\n')` ile temizlenir.

PYTHON

```
f = open('philosophers.txt', 'w')
f.write('John Locke\n')
f.write('David Hume\n')
f.close()

f = open('philosophers.txt', 'r')
line1 = f.readline()
line2 = f.readline()
f.close()
print(repr(line1))
print(line1.rstrip('\n'), '/', line2.rstrip('\n'))
print(repr(open('philosophers.txt').read()))
```

ÇIKTI

```
'John Locke\n'
John Locke / David Hume
'John Locke\nDavid Hume\n'
```

Ekleme ve sayısal veri (Slayt 15–16)

`'a'` modu eski içeriği korur. Dosyaya **yalnızca string** yazılabilir; sayılar önce `str()` ile çevrilir. Okunan sayılar da string'dir, `int()` / `float()` ile çevrilmelidir.

PYTHON

```
f = open('numbers.txt', 'w')
f.write(str(10) + '\n')
f.close()
f = open('numbers.txt', 'a')
f.write(str(20) + '\n')
f.close()
f = open('numbers.txt', 'r')
toplam = int(f.readline()) + int(f.readline())
f.close()
print(toplam)
```

ÇIKTI

```
30
```

Dosyayı döngüyle işlemek (Slayt 17–19)

Dosyada kaç öğe olduğu genellikle bilinmez, bu yüzden döngü kullanılır. `readline` dosya sonunda **boş string** döndürür; bu bir **sentinel** gibi kullanılabilir (`while line != ''`). Daha kısası, `for line in dosya:` dosya bitince kendiliğinden durur.

PYTHON

```
f = open('sales.txt', 'w')
for s in [1000.0, 2000.0, 3000.0]:
    f.write(str(s) + '\n')
f.close()

total = 0
f = open('sales.txt', 'r')
for line in f:
    total += float(line)
f.close()
print(total)
```

ÇIKTI

```
6000.0
```

Kayıtlar ve alanlar (Slayt 20–21)

Kayıt tek bir öğeyi tanımlayan veri kümesidir (ör. bir çalışan); **alan** kayıttaki tek veri parçasıdır (ad, kimlik no). Sıralı dosyada alanlar art arda yazılır, okurken bir kaydın tüm alanları sırayla okunur. Kayıtlarla ekleme, görüntüleme, arama, değiştirme ve silme yapılır.

PYTHON

```
f = open('employees.txt', 'w')
for ad, no in [('Ingrid', '4587'), ('Julia', '4588')]:
    f.write(ad + '\n')
    f.write(no + '\n')
f.close()

f = open('employees.txt', 'r')
name = f.readline()
while name != '':
    id_num = f.readline()
    print(name.rstrip('\n'), id_num.rstrip('\n'))
    name = f.readline()
f.close()
```

ÇIKTI

```
Ingrid 4587
Julia 4588
```

İstisnalar ve traceback (Slayt 22–23)

İstisna (exception), çalışma sırasında oluşan ve programı genellikle durduran hatadır. **Traceback** hatanın türünü, açıklamasını ve satırını gösterir. Bazı istisnalar dikkatli kodla önlenebilir (bölmeden önce sıfır kontrolü); bazıları önlenemez (sayı olmayan metni çevirmek, olmayan dosyayı açmak).

PYTHON
```
print(int('abc'))
```

ÇIKTI
```
ValueError: invalid literal for int() with base 10: 'abc'
```

PYTHON
```
f = open('olmayan_dosya.txt', 'r')
```

ÇIKTI
```
FileNotFoundError: [Errno 2] No such file or directory: 'olmayan_dosya.txt'
```

try / except (Slayt 24–27)

İstisna fırlatabilecek kod `try` bloğuna, **istisna işleyici** `except` bloğuna yazılır. `try` içinde istisna oluşursa:

- Türü `except`'te belirtilen türse o blok çalışır ve program `try/except`'ten sonra devam eder.
- Başka bir türse program traceback ile durur.

Birden çok tür için ayrı `except`'ler yazılır; tür belirtmeyen `except` her şeyi yakalar ve en sona yazılır. `except ValueError as err:` ile istisnanın mesajı alınabilir.

PYTHON
```
for metin in ['42', 'kırk', '0']:
    try:
        sonuc = 100 / int(metin)
        print(metin, '→', sonuc)
    except ValueError as err:
        print(metin, '→ sayı değil:', err)
    except ZeroDivisionError:
        print(metin, '→ sıfıra bölünemez')
print('Program devam ediyor')
```

ÇIKTI
```
42 → 2.380952380952381
kırk → sayı değil: invalid literal for int() with base 10: 'kırk'
0 → sıfıra bölünemez
```

ÇIKTI (DEVAM)

```
Program devam ediyor
```

else, finally ve işlenmeyen istisnalar (Slayt 28–31)

- `else` bloğu, `try` **istisnasız** biterse çalışır.
- `finally` bloğu istisna olsa da olmasa da **her zaman** çalışır; temizlik içindir (ör. dosyayı kapatmak).

İstisna, uygun türde `except` yoksa ya da `try`'ın dışında oluşmuşsa **işlenmemiş** kalır ve program durur.

PYTHON

```python
for deger in ['10', 'on']:
    try:
        sayi = int(deger)
    except ValueError:
        print(deger, '→ sayı değil')
    else:
        print(deger, '→', sayi * 2)
    finally:
        print('kontrol bitti')
```

ÇIKTI

```
10 → 20
kontrol bitti
on → sayı değil
kontrol bitti
```

Ek bilgi (resmi dokümantasyon): Dosyayı `with open(ad) as f:` ile açmak, blok bitince (hata olsa bile) dosyayı otomatik kapatır; `finally: f.close()` yazmanın kısa yoludur. Kaynak: docs.python.org/3/tutorial/inputoutput.html (Reading and Writing Files).

PYTHON

```python
with open('notlar.txt', 'w') as f:
    f.write('ilk satır\n')
with open('notlar.txt') as f:
    print(f.read().strip(), '| kapalı mı?', f.closed)
```

ÇIKTI

```
ilk satır | kapalı mı? False
```

Sık yapılan hatalar

- `'w'` **ile açarak eski içeriği silmek:** eklemek için `'a'` kullan.
- **Dosyaya sayı yazmak:** `f.write(10)` → `TypeError`; `str(10)` yaz.

PYTHON
```
f = open('rapor.txt', 'w')
f.write(10)
```

ÇIKTI
```
TypeError: write() argument must be str, not int
```

- **Okunan satırı sayı sanmak:** `'10\n'` bir string'dir; `int()` / `float()` gerekir.
- **Yanlış istisna türünü yakalamak:** `except ValueError` bir `ZeroDivisionError`'ı yakalamaz.
- **Dosyayı kapatmayı unutmak:** yazılanlar diske tam aktarılmayabilir; `close()` ya da `with` kullan.

Kısa tekrar soruları

1. `'w'` ve `'a'` modlarının farkı nedir?
2. `readline()` dosyanın sonuna gelince ne döndürür? Bu nasıl kullanılır?
3. Dosyadan okunan `'3.5\n'` satırıyla hesap yapmak için ne gerekir?
4. Kayıt ile alan arasındaki fark nedir?
5. `else` ve `finally` blokları hangi durumlarda çalışır?
6. Önlenebilen ve önlenemeyen birer istisna örneği ver.

Cevaplar

1. `'w'` dosya varsa içeriğini siler; `'a'` içeriği koruyup sonuna ekler.
2. Boş string (`''`) döndürür; `while line != ''` döngüsünde sentinel olarak kullanılır.
3. `float(satir)`; `float` sondaki `'\n'`'yi tolere eder, gerekirse önce `rstrip('\n')`.
4. Kayıt tek bir öğeyi tanımlayan veri kümesidir (bir çalışan); alan bu kaydın tek bir parçasıdır (adı).
5. `else` yalnızca `try` istisnasız bittiğinde; `finally` her durumda.
6. Önlenebilir: sıfıra bölme (öncesinde kontrol edilir). Önlenemez: kullanıcının sayı yerine metin girmesi ya da dosyanın olmaması.